ExInternBooks

AI-Powered Digital Library Platform

Project Documentation

Repository	yogap2232006-source/InternAssisment
Version	1.0.0
Stack	Next.js, Claude AI, RAG Vectors
Author	Yoga
Date	April 2026

1. Project Overview

ExInternBooks is an AI-powered digital library web application built during an internship assignment. It enables users to manage, explore, and interact with books using a Retrieval-Augmented Generation (RAG) chatbot powered by Claude AI and vector embeddings.

The platform allows users to upload books (PDF or TXT), browse a curated library, and ask natural-language questions about any book through an intelligent Library Assistant chatbot.

2. Key Features

2.1 Library Management

- View all books in a visually organized card-based grid layout
- Each card displays the book title, genre badge, description, and author
- Upload custom books in PDF or TXT format via the Upload Book button
- Auto Scrape Demo to automatically populate books from public sources

2.2 AI Library Assistant (Ask AI)

- Conversational chatbot interface powered by Claude AI
- RAG (Retrieval-Augmented Generation) with vector embeddings for accurate, context-aware answers
- Ask any question about books in the database and receive sourced answers
- Chat history retained within a session
- Exit Chat button to return to the main interface

2.3 Book Ingestion Pipeline

- Uploaded books are parsed and chunked into segments
- Each chunk is embedded as a vector and stored for semantic retrieval
- Supports both PDF and plain text (.txt) file formats

3. Technology Stack

Category	Technology
Frontend	Next.js (React) with Tailwind CSS
AI Engine	Claude AI (Anthropic) via API
RAG / Vectors	Vector embeddings for semantic book search
File Parsing	PDF and TXT ingestion pipeline
Hosting	Node.js server / Vercel (inferred)
Repo	GitHub - yogap2232006-source/InternAssisment

4. Project Structure

Below is the inferred directory structure of the ExInternBooks project:

```
InternAssisment/

  pages/              # Next.js pages (Library, Ask AI)

  components/         # Reusable UI components (BookCard, ChatUI)

  lib/                # Claude API integration, RAG utilities

  public/             # Static assets

  styles/             # Global CSS / Tailwind config

  api/                # API routes for book upload and chat

  README.md

  package.json
```

5. Application Screenshots

5.1 Library Page

The Library page displays all books in the database as interactive cards. Each card shows the book title, genre (as a colored badge), a short description, and the author. Users can upload new books or trigger an auto-scrape demo.

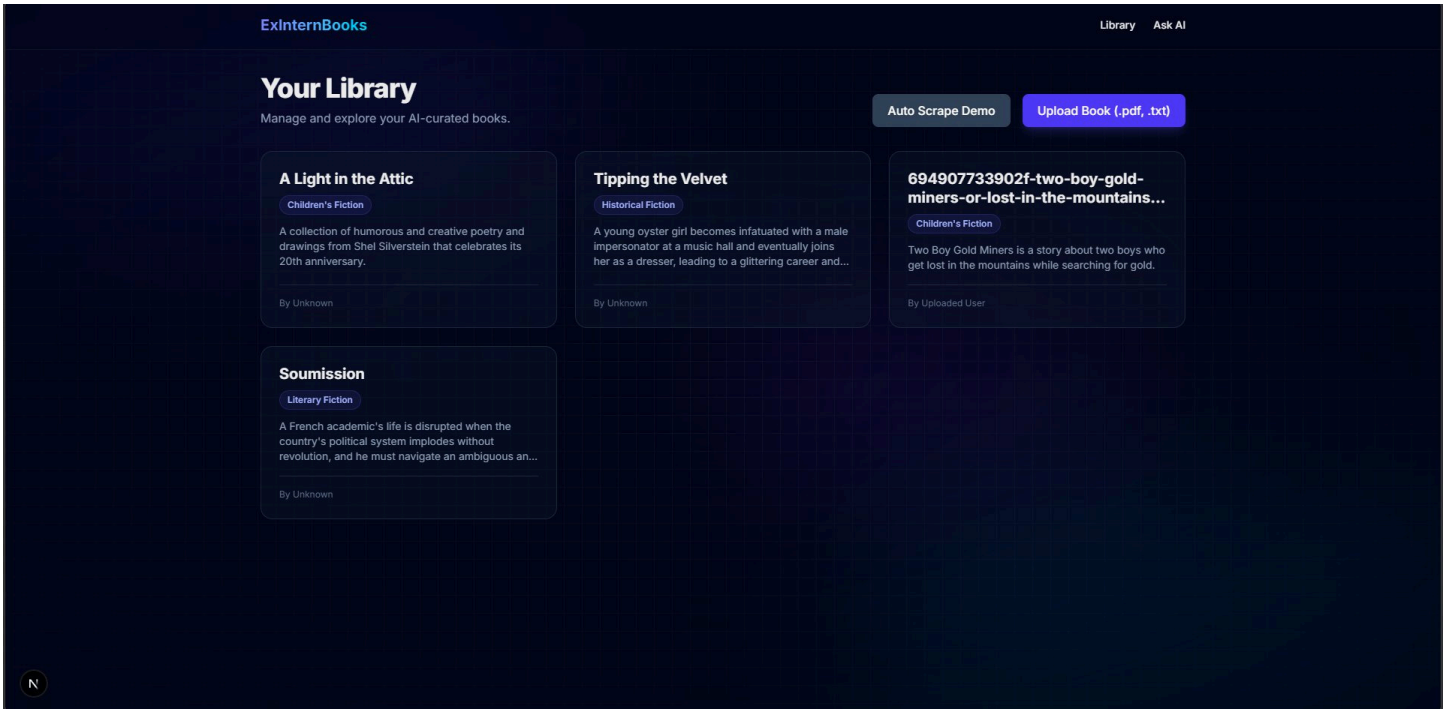

Figure 1 - Your Library: Book cards showing title, genre badge, description, and author

5.2 Ask AI - Empty State

The Ask AI page opens a full-screen chat interface. On first load, the AI Librarian greets the user and awaits a question. The interface is minimalist, with a text input at the bottom and an Exit Chat button to return to the library.

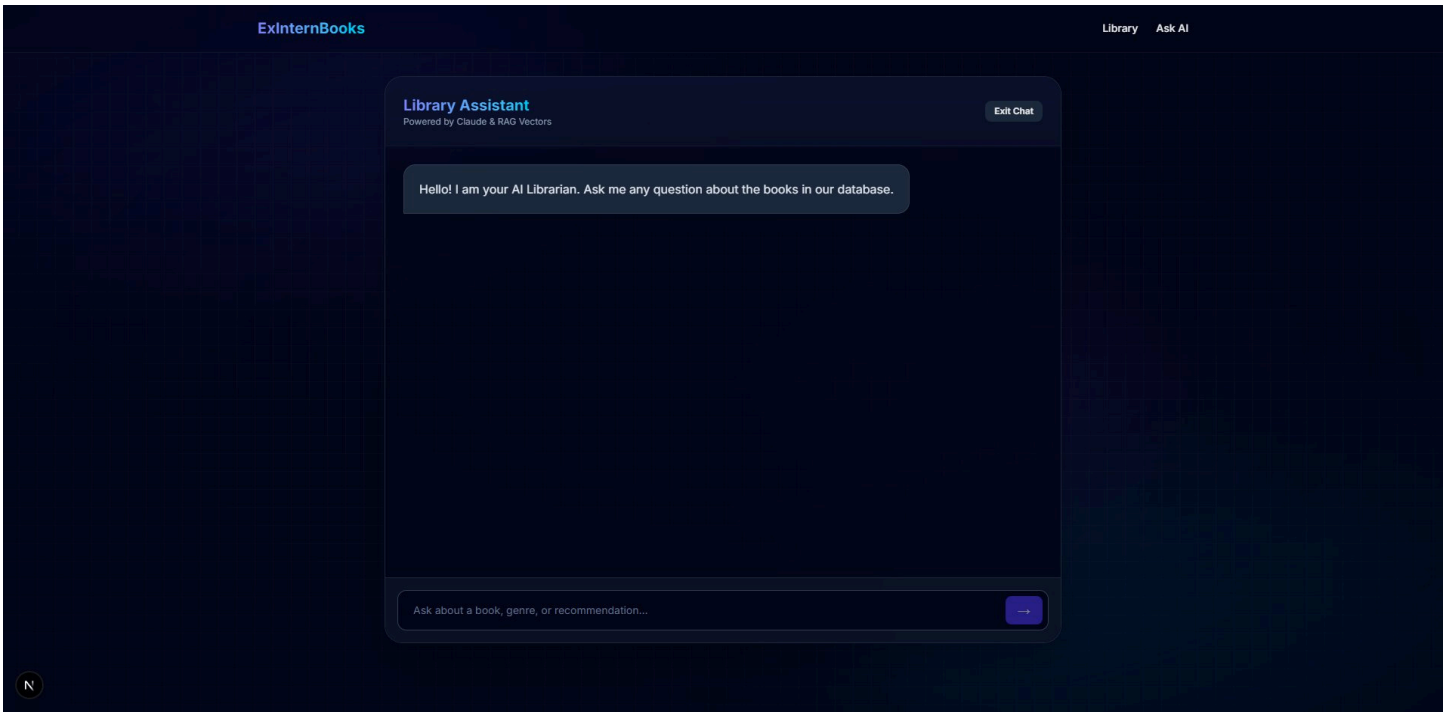

Figure 2 - Library Assistant: Initial empty chat state

5.3 Ask AI - Active Conversation

When the user sends a query, the chatbot responds using context retrieved from the vector database. The response includes source citations so users know which book and URL the information came from. This demonstrates the RAG pipeline in action.

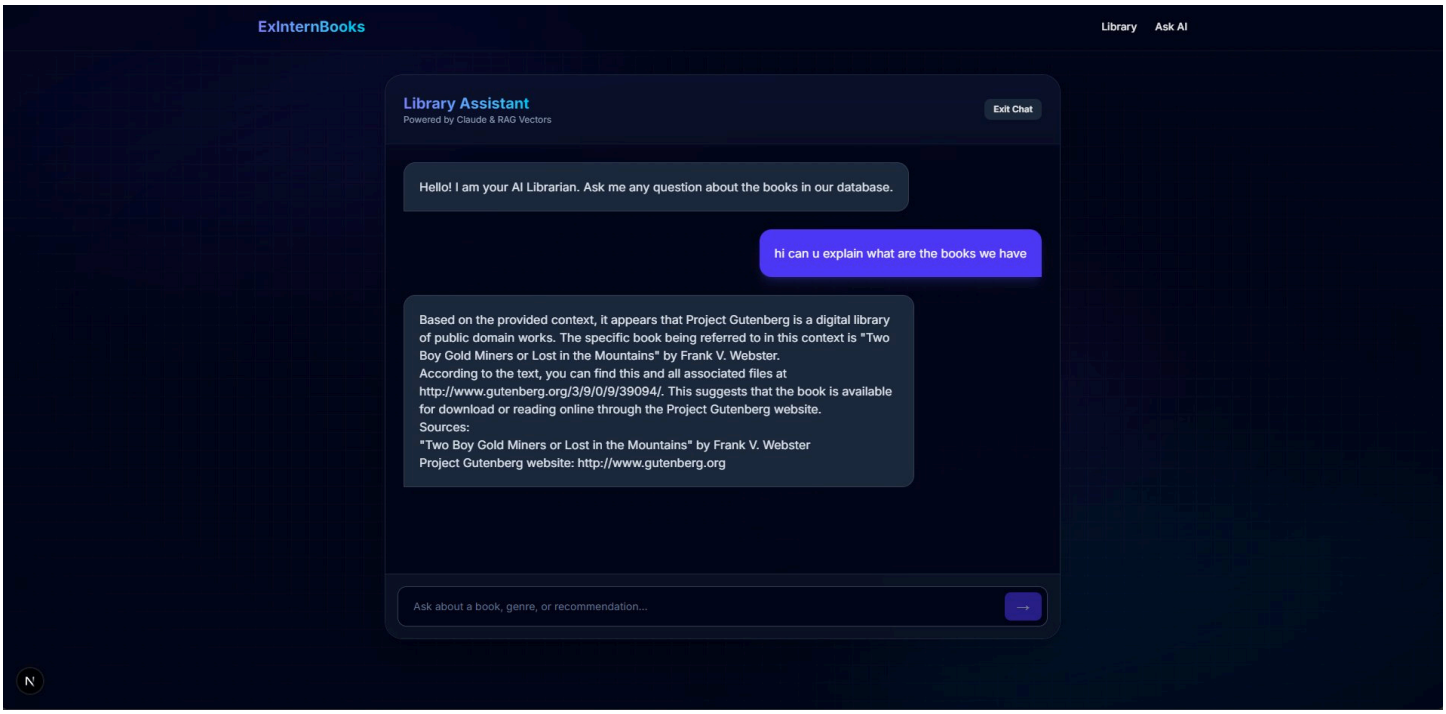

Figure 3 - Library Assistant: Active conversation with sourced AI response

6. Setup & Installation

6.1 Prerequisites

- Node.js v18 or higher
- npm or yarn package manager
- Anthropic API key (for Claude AI)
- Vector database credentials (if using external provider)

6.2 Installation Steps

1. Clone the repository

```
git clone https://github.com/yogap2232006-source/InternAssisment.git

cd InternAssisment
```

1. Install dependencies

```
npm install
```

1. Set up environment variables - create a .env.local file:

```
ANTHROPIC_API_KEY=your_api_key_here

VECTOR_DB_URL=your_vector_db_url
```

1. Run the development server

```
npm run dev
```

1. Open in browser

```
http://localhost:3000
```

7. How It Works

7.1 Book Upload Flow

1. User uploads a PDF or TXT file via the Upload Book button
2. The server parses and chunks the text content into segments
3. Claude AI generates embeddings (vectors) for each chunk
4. Vectors are stored in the vector database along with metadata
5. Book card is created and appears in the Library

7.2 RAG Chat Flow

1. User types a question in the Ask AI chat interface

2. The query is converted to a vector embedding
3. Semantically similar book chunks are retrieved from the vector store
4. Retrieved context + user query are sent to Claude AI
5. Claude generates a grounded response with source citations

8. API Endpoints

Method	Route	Description
GET	`/api/books`	Retrieve all books from the database
POST	`/api/upload`	Upload a PDF or TXT book, parse and vectorize it
POST	`/api/chat`	Send a message; retrieves context via RAG and calls Claude AI
POST	`/api/scrape`	Auto-scrape demo: populate library with sample books

9. Navigation & Pages

The application has two main pages accessible via the top navigation bar:

- Library - The home page displaying all books as cards with management controls
- Ask AI - Opens the AI Library Assistant chat interface

Both pages share a consistent dark-themed navigation bar with the ExInternBooks logo on the left.

10. Repository

The source code for this project is hosted on GitHub:

GitHub Repository: https://github.com/yogap2232006-source/InternAssisment

Clone command:

```
git clone https://github.com/yogap2232006-source/InternAssisment.git
```